

Lesson 20: Bootstrap and Permutation Methods for Two Samples

BSTA 551: Statistical Inference

Jessica Minnier

2026-03-09

Lesson 20: Bootstrap and Permutation Methods for Two Samples

Review: Lessons 18–19

Parametric two-sample methods so far:

- **Lesson 18:** Welch t / pooled t for $\mu_1 - \mu_2$; z -test for $p_1 - p_2$; F -test for σ_1^2 / σ_2^2
- **Lesson 19:** Paired t -test; variance reduction from positive within-pair correlation

All require **normality** (exact) or rely on the **CLT** (approximate). What if:

- Samples are small and distributions are visibly skewed?
- The parameter of interest is a **median, IQR, ratio**, or other non-mean statistic?
- Distributional assumptions are questionable?

Review: Lessons 18–19

Parametric two-sample methods so far:

- **Lesson 18:** Welch t / pooled t for $\mu_1 - \mu_2$; z -test for $p_1 - p_2$; F -test for σ_1^2 / σ_2^2
- **Lesson 19:** Paired t -test; variance reduction from positive within-pair correlation

All require **normality** (exact) or rely on the **CLT** (approximate). What if:

- Samples are small and distributions are visibly skewed?
- The parameter of interest is a **median, IQR, ratio**, or other non-mean statistic?
- Distributional assumptions are questionable?

Today's Goals:

1. Construct **two-sample bootstrap CIs** by resampling each group independently (C&H Ch. 3; DBC 10.6)
2. Perform **two-sample bootstrap hypothesis tests** by shifting to satisfy H_0 (C&H Ch. 8)
3. Implement the **two-sample permutation test** and understand exactly when it is valid (C&H Ch. 5)
4. Apply bootstrap CI and bootstrap test to **paired data**
5. Use the **infer** package for tidy, pipeline-based resampling inference

Roadmap

Method	Goal	Resamples From	Reference
Two-sample bootstrap CI	CI for $\theta_1 - \theta_2$	Each group separately	C&H Ch. 3; DBC 10.6
Two-sample bootstrap test	p-value for $H_0 : \theta_1 = \theta_2$	Shifted/centred samples	C&H Ch. 8
Two-sample permutation test	Test $H_0 : F_1 = F_2$	Pooled, relabeling	C&H Ch. 5
Paired bootstrap CI	CI for μ_D	Pairs with replacement	C&H Ch. 3
Paired bootstrap test	p-value for $H_0 : \mu_D = 0$	Shifted differences	C&H Ch. 8
infer package	Tidy pipeline for all of the above	—	infer vignettes

Roadmap

Method	Goal	Resamples From	Reference
Two-sample bootstrap CI	CI for $\theta_1 - \theta_2$	Each group separately	C&H Ch. 3; DBC 10.6
Two-sample bootstrap test	p-value for $H_0 : \theta_1 = \theta_2$	Shifted/centred samples	C&H Ch. 8
Two-sample permutation test	Test $H_0 : F_1 = F_2$	Pooled, relabeling	C&H Ch. 5
Paired bootstrap CI	CI for μ_D	Pairs with replacement	C&H Ch. 3
Paired bootstrap test	p-value for $H_0 : \mu_D = 0$	Shifted differences	C&H Ch. 8
infer package	Tidy pipeline for all of the above	—	infer vignettes

Motivating Example

Trauma registry: Compare hospital length of stay (LOS, days) between blunt and penetrating trauma patients. LOS is strongly right-skewed and n is moderate: the Welch t -test CI may be unreliable. Bootstrap and permutation give valid inference with no distributional assumptions.

Two-Sample Bootstrap CI

The Two-Sample Bootstrap Algorithm (C&H 3.2)

Setup: Independent samples $x_1, \dots, x_{n_1}$ from F_1 and $y_1, \dots, y_{n_2}$ from F_2 .

Goal: CI for $\theta = g(F_1, F_2)$, e.g., $\mu_1 - \mu_2$, $\text{median}_1 - \text{median}_2$, σ_1/σ_2 .

Algorithm: for $b = 1, \dots, B$:

1. Draw $x_1^{*b}, \dots, x_{n_1}^{*b}$ **with replacement** from $\{x_1, \dots, x_{n_1}\}$
2. Draw $y_1^{*b}, \dots, y_{n_2}^{*b}$ **with replacement** from $\{y_1, \dots, y_{n_2}\}$
3. Compute $\hat{\theta}^{*b} = g(\mathbf{x}^{*b}, \mathbf{y}^{*b})$

Percentile CI: $(\hat{\theta}_{(\alpha/2)}^*, \hat{\theta}_{(1-\alpha/2)}^*)$

Basic (reflection) CI: $(2\hat{\theta} - \hat{\theta}_{(1-\alpha/2)}^*, 2\hat{\theta} - \hat{\theta}_{(\alpha/2)}^*)$

Bootstrap SE CI: $\hat{\theta} \pm z_{\alpha/2} \cdot \widehat{\text{SE}}_{\text{boot}}$, where $\widehat{\text{SE}}_{\text{boot}} = \text{sd}(\hat{\theta}^{*1}, \dots, \hat{\theta}^{*B})$

The Two-Sample Bootstrap Algorithm (C&H 3.2)

Setup: Independent samples $x_1, \dots, x_{n_1}$ from F_1 and $y_1, \dots, y_{n_2}$ from F_2 .

Goal: CI for $\theta = g(F_1, F_2)$, e.g., $\mu_1 - \mu_2$, $\text{median}_1 - \text{median}_2$, σ_1/σ_2 .

Algorithm: for $b = 1, \dots, B$:

1. Draw $x_1^{*b}, \dots, x_{n_1}^{*b}$ **with replacement** from $\{x_1, \dots, x_{n_1}\}$
2. Draw $y_1^{*b}, \dots, y_{n_2}^{*b}$ **with replacement** from $\{y_1, \dots, y_{n_2}\}$
3. Compute $\hat{\theta}^{*b} = g(\mathbf{x}^{*b}, \mathbf{y}^{*b})$

Percentile CI: $(\hat{\theta}_{(\alpha/2)}^*, \hat{\theta}_{(1-\alpha/2)}^*)$

Basic (reflection) CI: $(2\hat{\theta} - \hat{\theta}_{(1-\alpha/2)}^*, 2\hat{\theta} - \hat{\theta}_{(\alpha/2)}^*)$

Bootstrap SE CI: $\hat{\theta} \pm z_{\alpha/2} \cdot \widehat{\text{SE}}_{\text{boot}}$, where $\widehat{\text{SE}}_{\text{boot}} = \text{sd}(\hat{\theta}^{*1}, \dots, \hat{\theta}^{*B})$

! Key Point

Resample each group **separately and independently** — this preserves the two-group structure and approximates the sampling distribution of $\hat{\theta}_1 - \hat{\theta}_2$. Pooling before resampling would impose H_0 : that is permutation, not bootstrap.

Medical Example: Trauma Length of Stay

► Code

Blunt: n = 40 mean = 7.86 median = 6.97 sd = 5.00

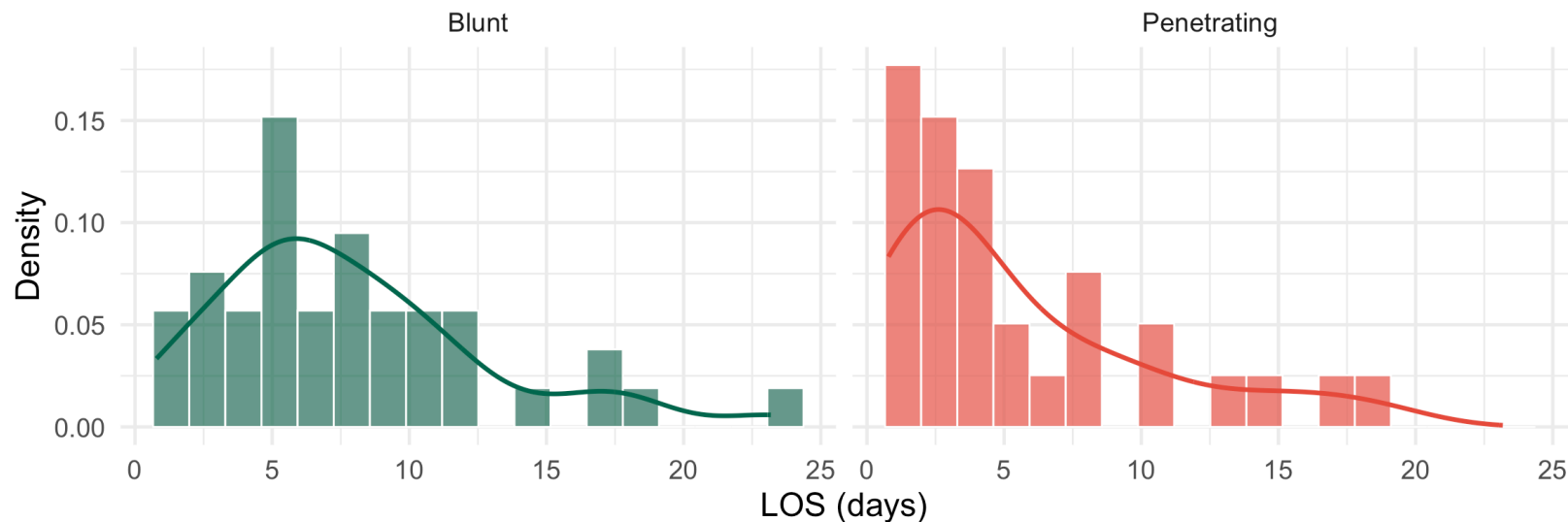
► Code

Penetrating: n = 30 mean = 5.62 median = 3.81 sd = 5.00

► Code

Hospital Length of Stay by Trauma Type

Right-skewed distributions — blunt trauma stays appear longer; evidence is borderline significant



Computing the Bootstrap CI: Difference in Means

```
1 set.seed(551)
2 B      <- 5000
3 n1     <- length(los_blunt)
4 n2     <- length(los_penet)
5 obs_diff <- mean(los_blunt) - mean(los_penet)
6
7 boot_diffs <- replicate(B, {
8   mean(sample(los_blunt, n1, replace = TRUE)) -
9   mean(sample(los_penet, n2, replace = TRUE))
10 })
11
12 ci_pct <- quantile(boot_diffs, c(0.025, 0.975))
13 ci_basic <- 2 * obs_diff - quantile(boot_diffs, c(0.975, 0.025))
14 ci_se <- obs_diff + c(-1, 1) * qnorm(0.975) * sd(boot_diffs)
15 ci_welch <- t.test(los_blunt, los_penet)$conf.int
16
17 cat(sprintf("Observed difference (Blunt - Penet): %.3f days\n", obs_diff))
```

Observed difference (Blunt - Penet): 2.235 days

```
1 cat(sprintf("Bootstrap SE: %.3f\n", sd(boot_diffs)))
```

Bootstrap SE: 1.191

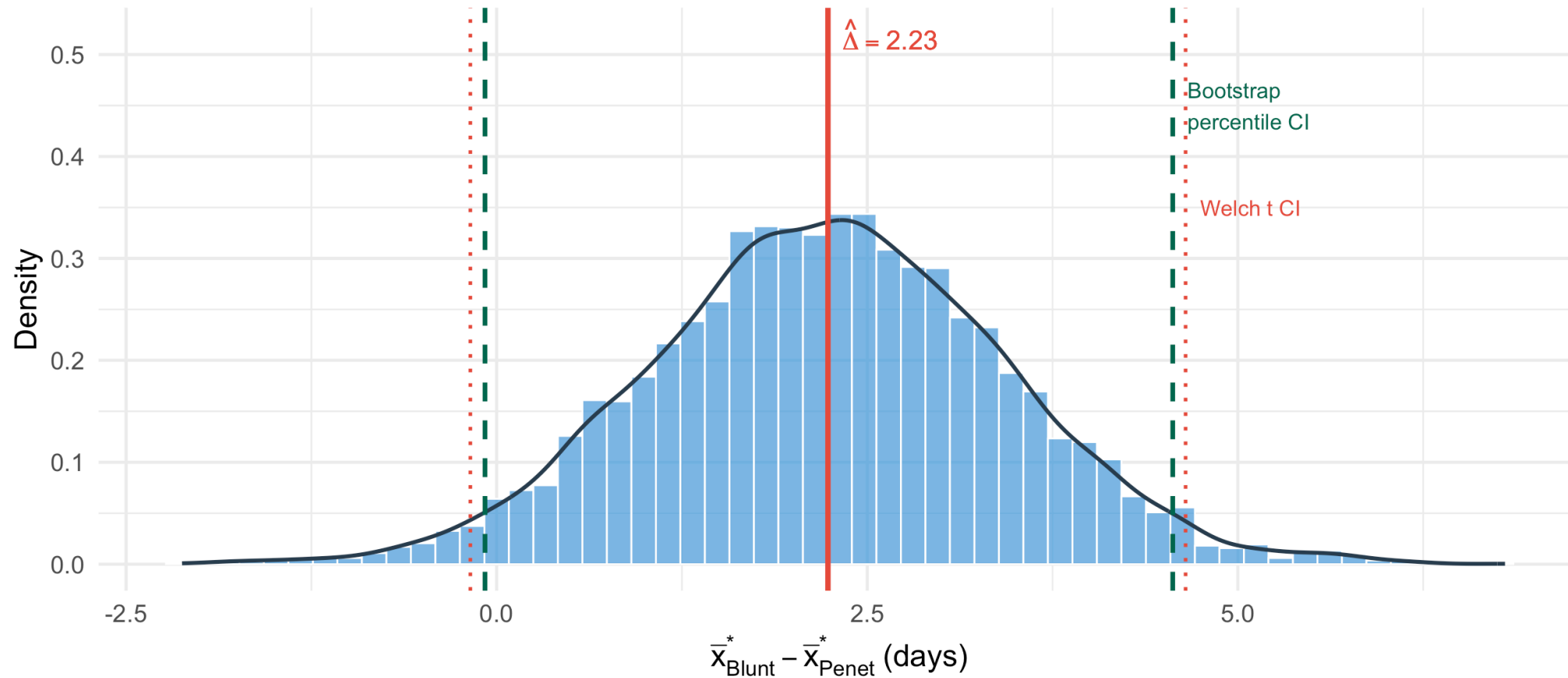
```
1 cat(sprintf("Bootstrap SE 95% CI: (%.3f, %.3f)\n", ci_se[1], ci_se[2]))
```

Visualizing the Bootstrap Distribution

► Code

Bootstrap Distribution: Mean LOS Difference (Blunt – Penetrating)

Green dashed = 95% bootstrap percentile CI | Red dotted = Welch t CI



Bootstrap CI for Any Statistic

A key advantage over parametric methods: the same algorithm works for **any statistic**.

```
1 set.seed(551)
2
3 # Difference in medians (no closed-form parametric CI exists)
4 boot_med <- replicate(B, {
5   median(sample(los_blunt, n1, replace = TRUE)) -
6   median(sample(los_penet, n2, replace = TRUE))
7 })
8
9 # Ratio of standard deviations
10 boot_sdr <- replicate(B, {
11   sd(sample(los_blunt, n1, replace = TRUE)) /
12   sd(sample(los_penet, n2, replace = TRUE))
13 })
14
15 cat("--- Difference in MEDIANS ---\n")
```

--- Difference in MEDIANS ---

```
1 cat(sprintf("  Observed: %.3f days\n",
2             median(los_blunt) - median(los_penet)))
```

Observed: 3.158 days

```
1 cat(sprintf("  Bootstrap 95% CI: (%.3f, %.3f)\n\n",
```

Two-Sample Bootstrap Hypothesis Test

Bootstrap Testing: The Core Challenge

Recall from Lesson 17: for a **one-sample** bootstrap test of $H_0 : \mu = \mu_0$, we **shift** the sample to satisfy H_0 before resampling, then compare the observed statistic to the shifted null distribution.

For **two samples**, we test $H_0 : \theta_1 = \theta_2$ (e.g., $\mu_1 = \mu_2$). The same logic applies: we must resample in a way that **imposes** H_0 .

Bootstrap Testing: The Core Challenge

Recall from Lesson 17: for a **one-sample** bootstrap test of $H_0 : \mu = \mu_0$, we **shift** the sample to satisfy H_0 before resampling, then compare the observed statistic to the shifted null distribution.

For **two samples**, we test $H_0 : \theta_1 = \theta_2$ (e.g., $\mu_1 = \mu_2$). The same logic applies: we must resample in a way that **imposes** H_0 .

⚠ Why Not Just Use the Bootstrap Distribution?

If we simply computed the distribution of $\hat{\theta}^* = \hat{\mu}_1^* - \hat{\mu}_2^*$ by resampling each group, we would get the **sampling distribution** centered at the **observed** $\hat{\Delta} = \hat{\mu}_1 - \hat{\mu}_2$.

This gives us a confidence interval — but **not a p-value under** H_0 . The null distribution must be centered at 0.

Two-Sample Bootstrap Test: The Shift Method (C&H 8.2)

Setup: Test $H_0 : \mu_1 = \mu_2$ vs $H_a : \mu_1 \neq \mu_2$.

Step 1: Impose H_0 by centering both samples at the same mean:

$$\tilde{x}_i = x_i - \bar{x} + \bar{X}_{\text{pool}}, \quad \tilde{y}_j = y_j - \bar{y} + \bar{X}_{\text{pool}}$$

where $\bar{X}_{\text{pool}} = \frac{n_1\bar{x} + n_2\bar{y}}{n_1 + n_2}$ is the pooled mean.

Now $\tilde{x}$ and $\tilde{y}$ have the **same mean** – H_0 is satisfied.

Two-Sample Bootstrap Test: The Shift Method (C&H 8.2)

Setup: Test $H_0 : \mu_1 = \mu_2$ vs $H_a : \mu_1 \neq \mu_2$.

Step 1: Impose H_0 by centering both samples at the same mean:

$$\tilde{x}_i = x_i - \bar{x} + \bar{X}_{\text{pool}}, \quad \tilde{y}_j = y_j - \bar{y} + \bar{X}_{\text{pool}}$$

where $\bar{X}_{\text{pool}} = \frac{n_1\bar{x} + n_2\bar{y}}{n_1 + n_2}$ is the pooled mean.

Now $\tilde{x}$ and $\tilde{y}$ have the **same mean** – H_0 is satisfied.

Step 2: Generate the null distribution by resampling from the shifted samples:

for $b = 1, \dots, B$:

1. Draw $\tilde{x}^{*b}$ (size n_1) from $\{\tilde{x}_i\}$ with replacement
2. Draw $\tilde{y}^{*b}$ (size n_2) from $\{\tilde{y}_j\}$ with replacement
3. Compute $T^{*b} = \bar{\tilde{x}}^{*b} - \bar{\tilde{y}}^{*b}$

p-value: $\hat{p} = \frac{1}{B} \sum_{b=1}^B \mathbf{1}(|T^{*b}| \geq |T_{\text{obs}}|)$

where $T_{\text{obs}} = \bar{x} - \bar{y}$.

Two-Sample Bootstrap Test: Implementation

```
1 set.seed(551)
2 B_test <- 9999
3
4 # Observed test statistic
5 t_obs <- mean(los_blunt) - mean(los_penet)
6
7 # Step 1: Shift samples to satisfy H0 (equal means)
8 grand_mean <- (sum(los_blunt) + sum(los_penet)) / (n1 + n2)
9 x_shift <- los_blunt - mean(los_blunt) + grand_mean
10 y_shift <- los_penet - mean(los_penet) + grand_mean
11
12 cat(sprintf("Before shifting: mean(blunt) = %.3f, mean(penet) = %.3f\n",
13             mean(los_blunt), mean(los_penet)))
```

Before shifting: mean(blunt) = 7.859, mean(penet) = 5.624

```
1 cat(sprintf("After shifting: mean(x_shift)= %.3f, mean(y_shift)= %.3f\n",
2             mean(x_shift), mean(y_shift)))
```

After shifting: mean(x_shift)= 6.901, mean(y_shift)= 6.901

```
1 # Step 2: Resample from shifted samples
2 boot_null <- replicate(B_test, {
3   mean(sample(x_shift, n1, replace = TRUE)) -
4   mean(sample(y_shift, n2, replace = TRUE))
5 }
```

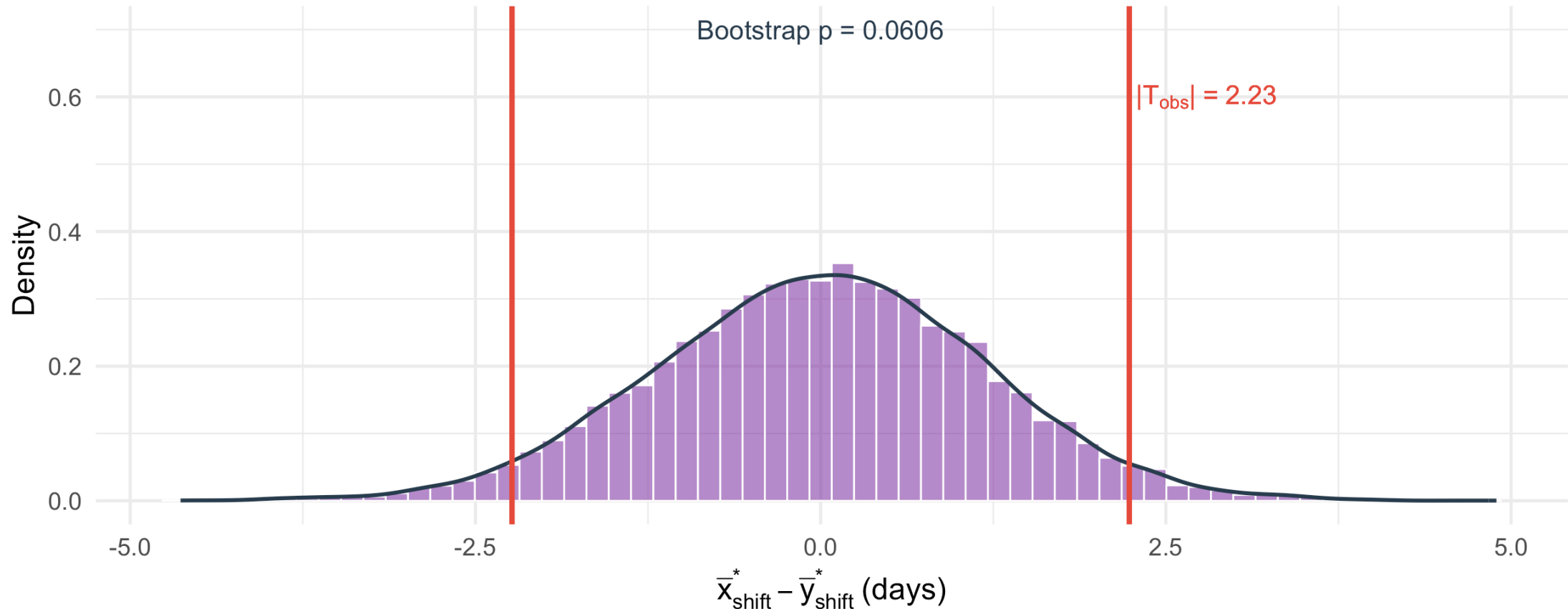


Visualizing the Bootstrap Null Distribution

► Code

Bootstrap Null Distribution: Difference in Mean LOS (Shift Method)

Null distribution centered at 0 by shifting both samples to grand mean



Bootstrap Test for Any Statistic (e.g., Median)

The shift method generalizes to **any location statistic** – shift both samples to equal medians (or equal means) and resample.

```
1 set.seed(551)
2
3 # Bootstrap test for MEDIAN difference
4 t_obs_med <- median(los_blunt) - median(los_penet)
5
6 # Shift both samples to have equal medians (use grand median or grand mean)
7 x_shift_med <- los_blunt - median(los_blunt) + grand_mean
8 y_shift_med <- los_penet - median(los_penet) + grand_mean
9
10 boot_null_med <- replicate(B_test, {
11   median(sample(x_shift_med, n1, replace = TRUE)) -
12   median(sample(y_shift_med, n2, replace = TRUE))
13 })
14
15 p_boot_med <- mean(abs(boot_null_med) >= abs(t_obs_med))
16
17 cat(sprintf("Median difference: %.3f days\n", t_obs_med))
```

Median difference: 3.158 days

```
1 cat(sprintf("Bootstrap test p (median): %.4f\n", p_boot_med))
```



Bootstrap Test for Any Statistic (e.g., Median)

The shift method generalizes to **any location statistic** – shift both samples to equal medians (or equal means) and resample.

```
1 set.seed(551)
2
3 # Bootstrap test for MEDIAN difference
4 t_obs_med <- median(los_blunt) - median(los_penet)
5
6 # Shift both samples to have equal medians (use grand median or grand mean)
7 x_shift_med <- los_blunt - median(los_blunt) + grand_mean
8 y_shift_med <- los_penet - median(los_penet) + grand_mean
9
10 boot_null_med <- replicate(B_test, {
11   median(sample(x_shift_med, n1, replace = TRUE)) -
12   median(sample(y_shift_med, n2, replace = TRUE))
13 })
14
15 p_boot_med <- mean(abs(boot_null_med) >= abs(t_obs_med))
16
17 cat(sprintf("Median difference: %.3f days\n", t_obs_med))
```

Median difference: 3.158 days

```
1 cat(sprintf("Bootstrap test p (median): %.4f\n", p_boot_med))
```



Two-Sample Permutation Test

The Permutation Test: Core Idea (C&H Ch. 5)

Null hypothesis: $H_0 : F_1 = F_2$ — the two groups have the **same distribution**.

Under H_0 , the group label on each observation is **arbitrary** (exchangeable). Any assignment of the $N = n_1 + n_2$ pooled observations into groups of size n_1 and n_2 is equally likely.

The Permutation Test: Core Idea (C&H Ch. 5)

Null hypothesis: $H_0 : F_1 = F_2$ — the two groups have the **same distribution**.

Under H_0 , the group label on each observation is **arbitrary** (exchangeable). Any assignment of the $N = n_1 + n_2$ pooled observations into groups of size n_1 and n_2 is equally likely.

Algorithm: for $b = 1, \dots, B$:

1. Pool all N observations
2. Randomly permute labels: assign the first n_1 to group 1, the rest to group 2
3. Compute T^{*b} (the test statistic under the permuted labeling)

Two-sided p-value: $\hat{p} = \frac{1}{B} \sum_{b=1}^B \mathbf{1}(|T^{*b}| \geq |T_{\text{obs}}|)$

The Permutation Test: Core Idea (C&H Ch. 5)

Null hypothesis: $H_0 : F_1 = F_2$ – the two groups have the **same distribution**.

Under H_0 , the group label on each observation is **arbitrary** (exchangeable). Any assignment of the $N = n_1 + n_2$ pooled observations into groups of size n_1 and n_2 is equally likely.

Algorithm: for $b = 1, \dots, B$:

- 1. Pool all N observations
- 2. Randomly permute labels: assign the first n_1 to group 1, the rest to group 2
- 3. Compute T^{*b} (the test statistic under the permuted labeling)

Two-sided p-value: $\hat{p} = \frac{1}{B} \sum_{b=1}^B \mathbf{1}(|T^{*b}| \geq |T_{\text{obs}}|)$

! Bootstrap vs. Permutation – The Essential Distinction			
	Bootstrap CI	Bootstrap Test	Permutation Test
Purpose	Estimate variability	Test H_0	Test H_0
Resamples from	Each group separately	Shifted samples	Pooled (relabeling)
Approximates	Sampling dist. of $\hat{\theta}$	Null dist. of T	Null dist. of T
H_0 imposed?	No	Yes (via shift)	Yes ($F_1 = F_2$)
Assumption	Independence	Independence	Exchangeability

When Is the Permutation Test Valid?

The permutation test is valid (exact Type I error control) when the observations are **exchangeable** under H_0 .

! Exchangeability: What It Means

Observations $X_1, \dots, X_{n_1}, Y_1, \dots, Y_{n_2}$ are **exchangeable** under H_0 if permuting group labels does not change the joint distribution. This holds when:

$$H_0 : F_1 = F_2 \quad (\text{identical distributions})$$

If H_0 is true, any relabeling of the pooled data is just as likely as any other — permuting is valid.

When Is the Permutation Test Valid?

The permutation test is valid (exact Type I error control) when the observations are **exchangeable** under H_0 .

⚠ Exchangeability: What It Means

Observations $X_1, \dots, X_{n_1}, Y_1, \dots, Y_{n_2}$ are **exchangeable** under H_0 if permuting group labels does not change the joint distribution. This holds when:

$$H_0 : F_1 = F_2 \quad (\text{identical distributions})$$

If H_0 is true, any relabeling of the pooled data is just as likely as any other — permuting is valid.

When exchangeability holds:

- Randomized experiments: treatment assigned at random, so under H_0 labels are arbitrary
- Observational data: valid if groups differ **only** in the parameter of interest (strong assumption)
- Testing $H_0 : F_1 = F_2$ exactly (not just equal means)

When exchangeability fails (permutation test is invalid):

- Groups differ in **variance or shape** but not in means: permuting is not valid for a test of means because $F_1 \neq F_2$ even under the mean-equality H_0
- Paired or clustered data: pairing creates dependence — don't ignore the structure
- Confounding: if groups also differ on covariates, labels are not exchangeable

Exchangeability in Practice: What Hypothesis Are We Testing?

A Subtle Point

The permutation test is an **exact test** of $H_0 : F_1 = F_2$. When used with the mean difference as the test statistic, it **also** has good power to detect $\mu_1 \neq \mu_2$.

However, if the groups differ in **variance** but not in mean, the permutation test (using the mean difference as the statistic) can have **inflated Type I error** because the null $F_1 = F_2$ includes equal variances. Rejecting $H_0 : F_1 = F_2$ does not tell us *why* it was rejected.

Solutions:

- If you care only about means: use the Welch t -test (robust to unequal variances) or the bootstrap test (shift method)
- If you want a permutation test for means when variances may differ: use a standardized test statistic (Welch-type) but permute — this has approximately valid Type I error
- Reserve permutation tests for settings where $F_1 = F_2$ is a plausible and scientifically meaningful null

Exchangeability: Randomized vs. Observational Studies

► Code

Permutation Test: Validity by Setting

Setting	Exchangeable?	Valid?	Recommendation
RCT: treatment vs. placebo (randomized assignment)	Yes	Yes (exact)	Permutation valid
Observational: blunt vs. penetrating trauma	Approximately	Approximately	Permutation OK; Welch t also reasonable
Groups differ in variance but not in mean	No	No (inflated Type I)	Bootstrap test or Welch t
Paired data treated as independent (wrong!)	No	No (ignores structure)	Pair the data; use paired bootstrap or paired t-test

Permutation Test: Implementation

```
1 set.seed(551)
2 B_perm <- 9999
3 all_los <- c(los_blunt, los_penet)
4 N <- length(all_los)
5 t_obs <- mean(los_blunt) - mean(los_penet)
6
7 perm_diffs <- replicate(B_perm, {
8   idx <- sample(N)
9   mean(all_los[idx[seq_len(n1)]])) - mean(all_los[idx[(n1 + 1):N]])
10 })
11
12 p_perm <- mean(abs(perm_diffs) >= abs(t_obs))
13 p_welch <- t.test(los_blunt, los_penet)$p.value
14
15 cat(sprintf("Observed mean difference:   %.3f days\n", t_obs))
```

Observed mean difference: 2.235 days

```
1 cat(sprintf("Permutation p-value:      %.4f\n",      p_perm))
```

Permutation p-value: 0.0702

```
1 cat(sprintf("Welch t-test p-value:      %.4f\n",      p_welch))
```

Welch t-test p-value: 0.0688

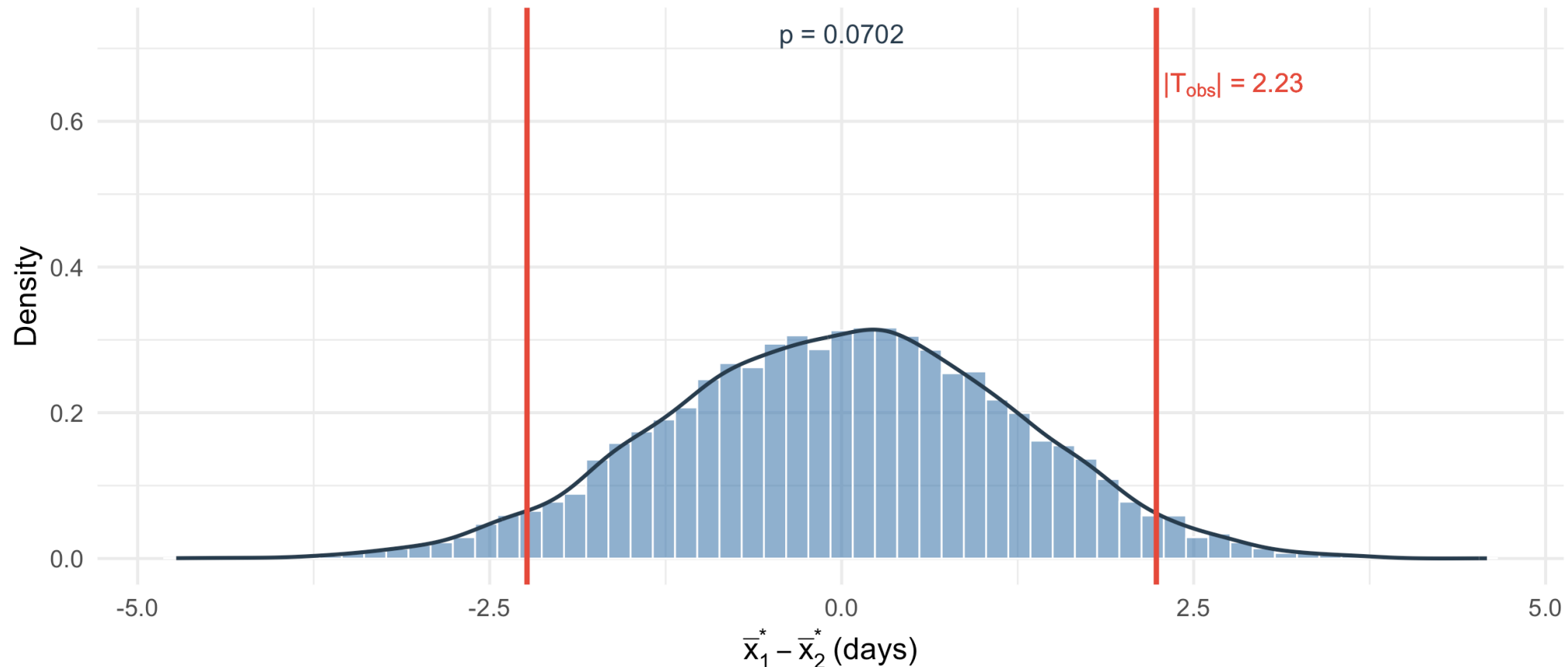
```
1 cat(sprintf("Bootstrap test p-value:      %.4f\n",      p_boot))
```

Visualizing the Permutation Null Distribution

► Code

Permutation Null Distribution: Difference in Mean LOS

Red lines mark $\pm|T_{\text{obs}}|$. Fraction of permuted statistics beyond red = p-value.



Permutation Tests Work with Any Statistic

```
1 set.seed(551)
2
3 # Permutation test for difference in MEDIANs
4 t_obs_med <- median(los_blunt) - median(los_penet)
5 perm_med  <- replicate(9999, {
6   idx <- sample(N)
7   median(all_los[idx[seq_len(n1)]] - median(all_los[idx[(n1 + 1):N]]))
8 })
9 p_perm_med <- mean(abs(perm_med) >= abs(t_obs_med))
10
11 cat(sprintf("Median difference: %.3f    permutation p = %.4f\n",
12            t_obs_med, p_perm_med))
```

Median difference: 3.158 permutation p = 0.0184

```
1 # Permutation test for VARIANCE RATIO (vs F-test)
2 t_obs_var <- var(los_blunt) / var(los_penet)
3 perm_var  <- replicate(9999, {
4   idx <- sample(N)
5   var(all_los[idx[seq_len(n1)]] / var(all_los[idx[(n1 + 1):N]]))
6 })
7 p_perm_var <- mean(perm_var >= t_obs_var | perm_var <= 1 / t_obs_var)
8 p_ftest    <- var.test(los_blunt, los_penet)$p.value
9
```



Permutation Tests Work with Any Statistic

```
1 set.seed(551)
2
3 # Permutation test for difference in MEDIANs
4 t_obs_med <- median(los_blunt) - median(los_penet)
5 perm_med  <- replicate(9999, {
6   idx <- sample(N)
7   median(all_los[idx[seq_len(n1)]] - median(all_los[idx[(n1 + 1):N]]))
8 })
9 p_perm_med <- mean(abs(perm_med) >= abs(t_obs_med))
10
11 cat(sprintf("Median difference: %.3f    permutation p = %.4f\n",
12            t_obs_med, p_perm_med))
```

Median difference: 3.158 permutation p = 0.0184

```
1 # Permutation test for VARIANCE RATIO (vs F-test)
2 t_obs_var <- var(los_blunt) / var(los_penet)
3 perm_var  <- replicate(9999, {
4   idx <- sample(N)
5   var(all_los[idx[seq_len(n1)]] / var(all_los[idx[(n1 + 1):N]]))
6 })
7 p_perm_var <- mean(perm_var >= t_obs_var | perm_var <= 1 / t_obs_var)
8 p_ftest    <- var.test(los_blunt, los_penet)$p.value
9
```



Bootstrap for Paired Data

Paired Data: Setup and the Crossover Trial Example

Paired data arises when each subject provides two measurements:

- Crossover trials: same patient receives both treatments (in random order)
- Before-after studies: same patient measured at two time points
- Matched cohort studies: pairs matched on confounders

The natural estimand is $\mu_D = E[D_i]$ where $D_i = X_{i,\text{post}} - X_{i,\text{pre}}$ (or Treatment – Control).

► Code

Mean difference (Drug – Placebo): -2.92 mmHg

► Code

SD of differences: 6.27 mmHg

► Code

Paired t-test p-value: 0.0286

Paired Bootstrap CI

For paired data, the bootstrap resamples **entire pairs** (D_i) with replacement, which preserves the pairing structure.

Algorithm: for $b = 1, \dots, B$:

1. Draw $D_1^{*b}, \dots, D_n^{*b}$ **with replacement** from $\{D_1, \dots, D_n\}$
2. Compute $\bar{D}^{*b} = \frac{1}{n} \sum D_i^{*b}$

Percentile CI: $(\bar{D}_{(\alpha/2)}^*, \bar{D}_{(1-\alpha/2)}^*)$

No normality assumption on D_i .

```
1 d_cross <- bp_data$diff
2
3 set.seed(551)
4 boot_D <- replicate(5000, mean(sample(d_cross, n_pair, replace = TRUE)))
5 ci_boot_paired <- quantile(boot_D, c(0.025, 0.975))
6 ci_t_paired <- t.test(d_cross)$conf.int
7
8 cat(sprintf("Observed mean difference:      %.3f mmHg\n", mean(d_cross)))
```

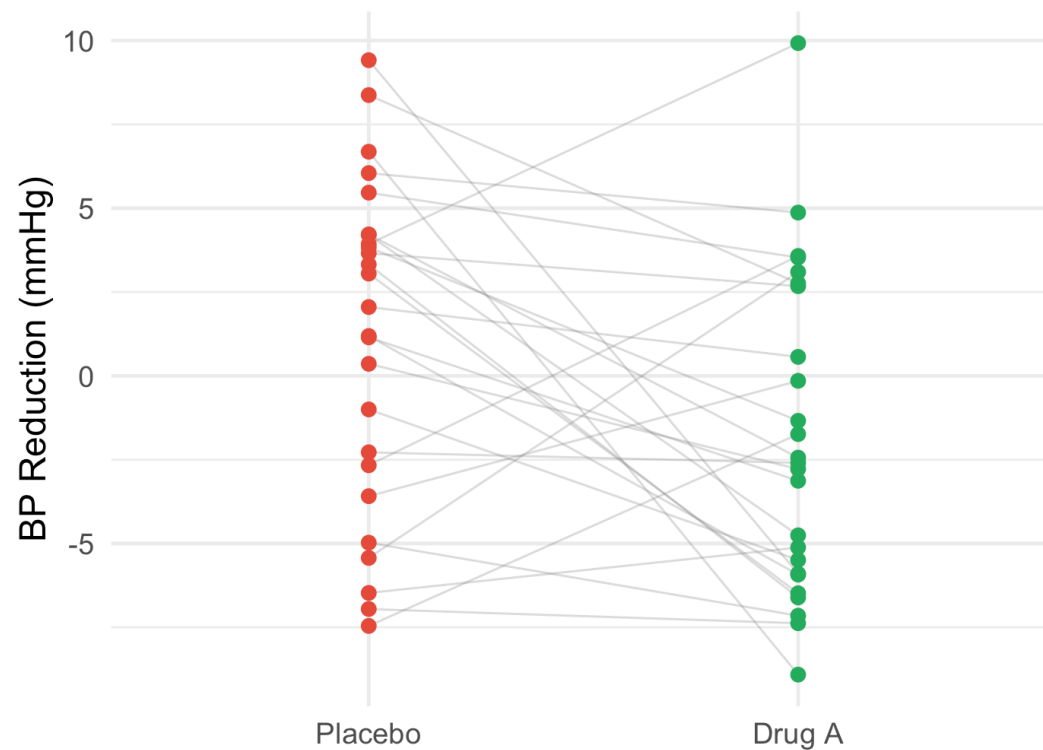
Observed mean difference: -2.921 mmHg

```
1 cat(sprintf("Bootstrap SE:                  %.3f\n", sd(boot_D)))
```

Visualizing the Paired Bootstrap Distribution

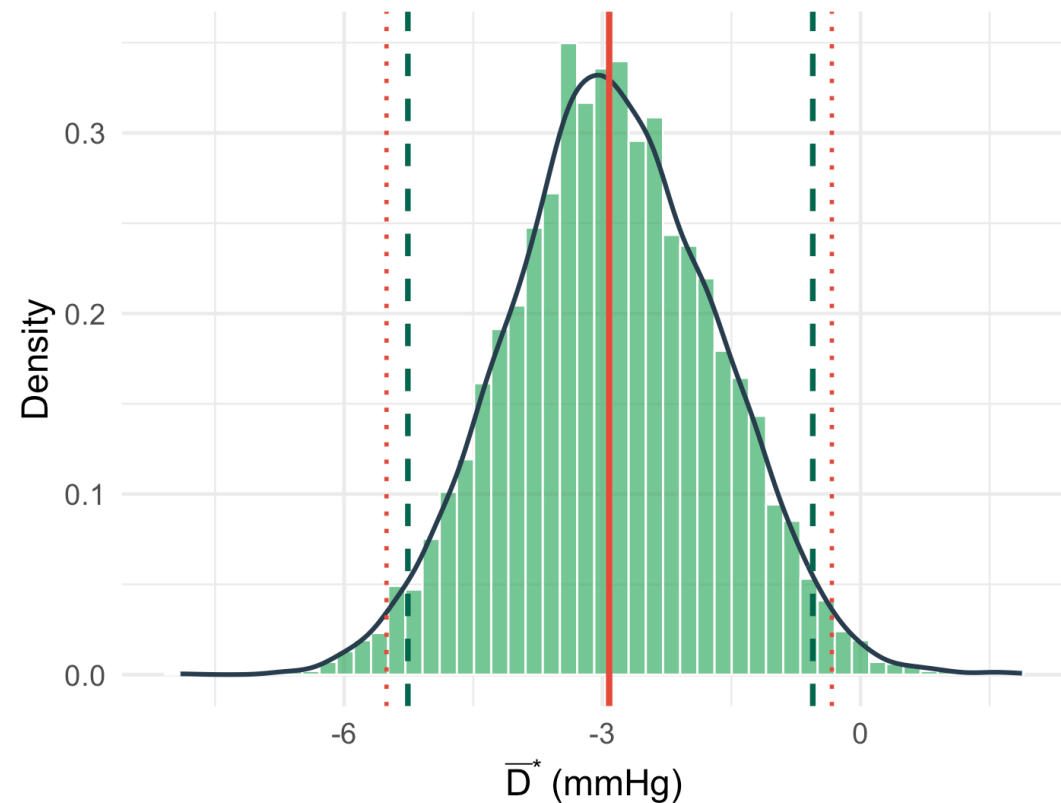
► Code

Crossover Trial: BP Reduction per Patient



Paired Bootstrap Distribution: Mean BP Reduction

Green dashed = bootstrap percentile CI
Red dotted = paired t CI



Paired Bootstrap Test

To test $H_0 : \mu_D = 0$, we must shift the differences to satisfy H_0 :

$$\tilde{D}_i = D_i - \bar{D}$$

Now $\{\tilde{D}_i\}$ have mean exactly 0 – H_0 is imposed. We then resample from the shifted differences.

Algorithm: for $b = 1, \dots, B$:

1. Draw $\tilde{D}_1^{*b}, \dots, \tilde{D}_n^{*b}$ with replacement from $\{\tilde{D}_i\}$
2. Compute $T^{*b} = \bar{\tilde{D}}^{*b}$

p-value: $\hat{p} = \frac{1}{B} \sum_{b=1}^B \mathbf{1}(|T^{*b}| \geq |\bar{D}_{\text{obs}}|)$

```
1 set.seed(551)
2 B_paired <- 9999
3 D_obs <- mean(d_cross)
4
5 # Shift differences to mean 0 (imposing H0)
6 d_shifted <- d_cross - mean(d_cross)
7
8 cat(sprintf("Before shifting: mean(D) = %.4f\n", mean(d_cross)))
```

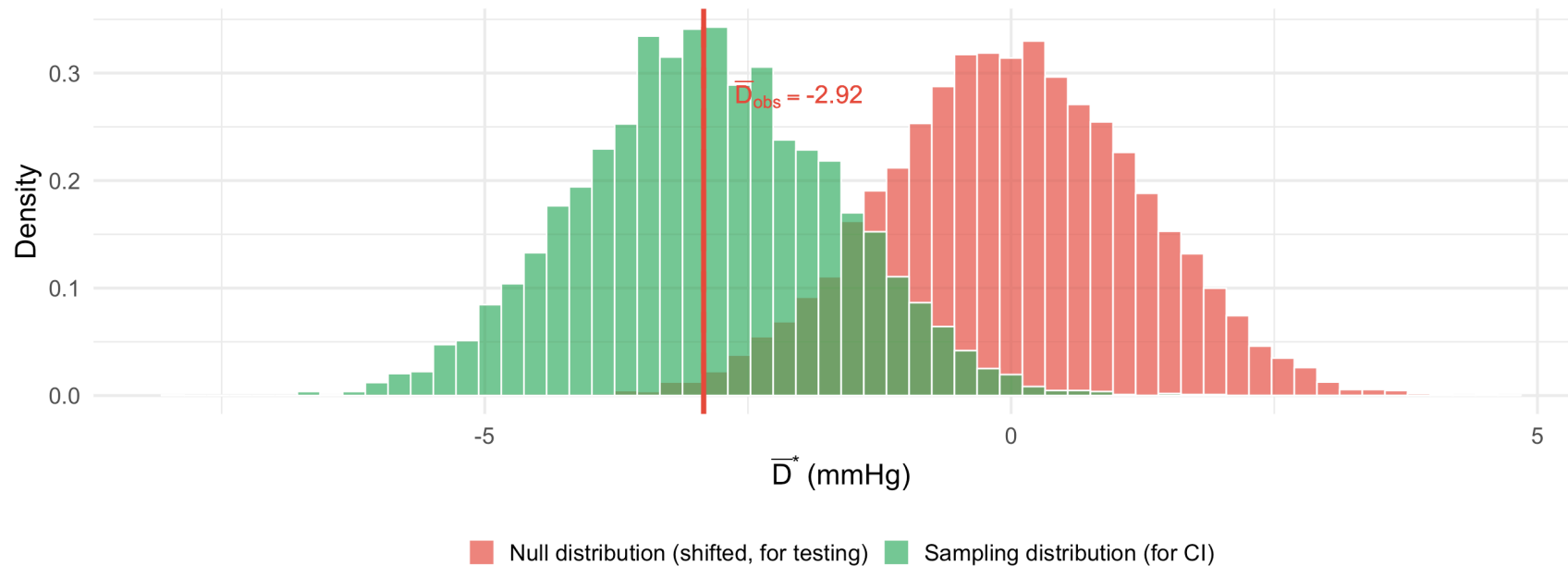
Before shifting: mean(D) = -2.9206

Paired Bootstrap Null vs. Sampling Distribution

► Code

Paired Bootstrap: Null Distribution vs. Sampling Distribution

Red = shifted to H_0 (for p-value) | Green = resampled as-is (for CI)



i Note

The null distribution (red) is centered at 0 — it was shifted to impose $H_0 : \mu_D = 0$. The sampling distribution (green) is centered at $\bar{D}_{obs}$ and is used for the CI, not the test. Because the true effect is modest (~ -3 mmHg with substantial within-subject variability), the two distributions overlap considerably, mirroring the borderline p-value.

The **infer** Package

The `infer` Package: Tidy Resampling Inference

The `infer` package (part of the `tidymodels` ecosystem) provides a **unified, pipe-friendly framework** for hypothesis testing and interval estimation via simulation.

The workflow follows four consistent verbs:

Step	Verb	What it does
1	<code>specify()</code>	Declare the response (and optional explanatory) variable
2	<code>hypothesize()</code>	State the null hypothesis (for tests only)
3	<code>generate()</code>	Generate bootstrap samples or permutations
4	<code>calculate()</code>	Compute the statistic of interest

Then use `get_p_value()`, `get_confidence_interval()`, or `visualize()` to summarize.

The `infer` Package: Tidy Resampling Inference

The `infer` package (part of the `tidymodels` ecosystem) provides a **unified, pipe-friendly framework** for hypothesis testing and interval estimation via simulation.

The workflow follows four consistent verbs:

Step	Verb	What it does
1	<code>specify()</code>	Declare the response (and optional explanatory) variable
2	<code>hypothesize()</code>	State the null hypothesis (for tests only)
3	<code>generate()</code>	Generate bootstrap samples or permutations
4	<code>calculate()</code>	Compute the statistic of interest

Then use `get_p_value()`, `get_confidence_interval()`, or `visualize()` to summarize.

Why Use `infer`?

- Consistent syntax across one-sample, two-sample, and paired settings
- Integrates naturally with `dplyr` pipelines and tidy data
- Makes the conceptual steps explicit: `specify` → `hypothesize` → `generate` → `calculate`
- Excellent for teaching and reproducible research workflows



Setting Up the Data for `infer`

The `infer` package requires **tidy (long) data** — one row per observation.

```
1 # Two-sample LOS data in tidy format
2 los_tidy <- tibble(
3   LOS    = c(los_blunt, los_penet),
4   Group  = rep(c("Blunt", "Penetrating"),
5               c(length(los_blunt), length(los_penet)))
6 )
7
8 los_tidy |> slice_sample(n = 6)
```

```
# A tibble: 6 × 2
  LOS Group
<dbl> <chr>
1 12.3 Blunt
2  5.03 Blunt
3  2.09 Penetrating
4  8.52 Blunt
5  2.63 Penetrating
6  2.87 Blunt
```

```
1 # Paired blood pressure data in tidy format
2 bp_tidy <- bp_data |>
3   select(patient, diff) # use the already-computed differences
4
```



Two-Sample Permutation Test with `infer`

```
1 set.seed(551)
2
3 # Step 1-2: Specify and hypothesize
4 # Step 3: Generate permutation null distribution
5 null_dist_perm <- los_tidy |>
6   specify(LOS ~ Group) |>
7   hypothesize(null = "independence") |>
8   generate(reps = 9999, type = "permute") |>
9   calculate(stat = "diff in means", order = c("Blunt", "Penetrating"))
10
11 # Observed statistic
12 obs_stat <- los_tidy |>
13   specify(LOS ~ Group) |>
14   calculate(stat = "diff in means", order = c("Blunt", "Penetrating"))
15
16 obs_stat
```

Response: LOS (numeric)

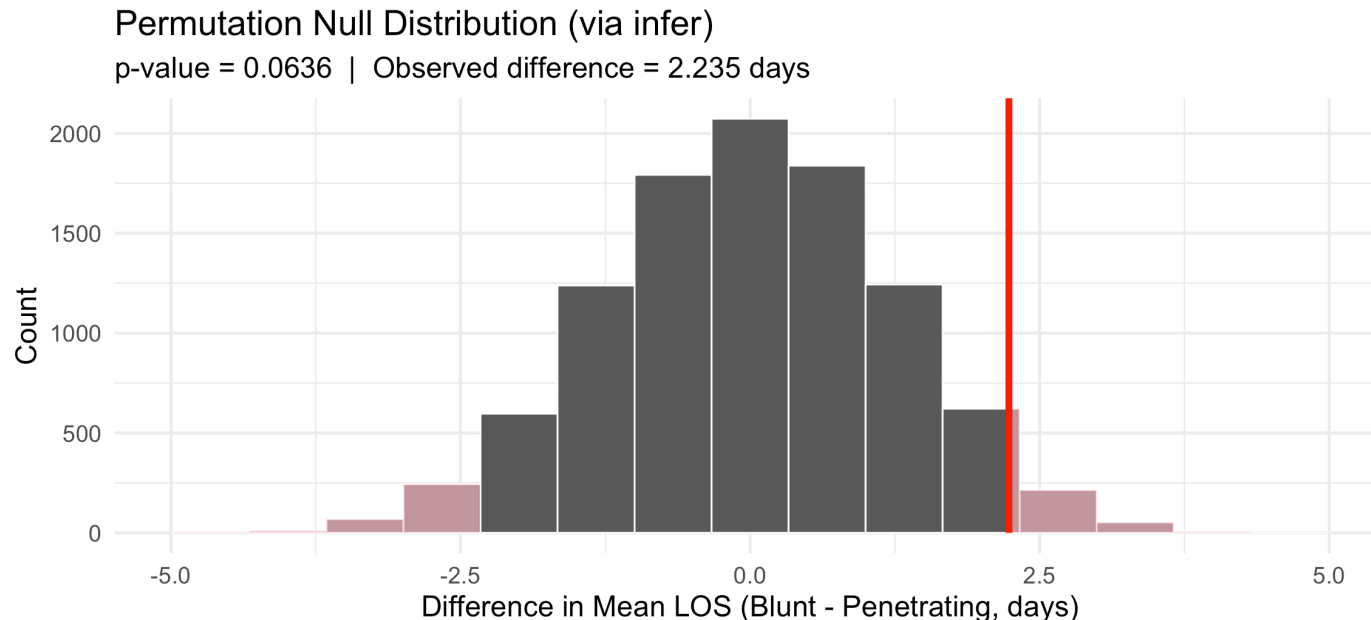
Explanatory: Group (factor)

A tibble: 1 × 1

	stat
	<dbl>
1	2.23

Visualizing the `infer` Permutation Distribution

```
1 visualize(null_dist_perm) +  
2   shade_p_value(obs_stat, direction = "two-sided") +  
3   labs(  
4     title      = "Permutation Null Distribution (via infer)",  
5     subtitle   = sprintf("p-value = %.4f | Observed difference = %.3f days",  
6                           pval_perm$p_value, obs_stat$stat),  
7     x          = "Difference in Mean LOS (Blunt - Penetrating, days)",  
8     y          = "Count"  
9   )
```



Two-Sample Bootstrap CI with `infer`

```
1 set.seed(551)
2
3 # Bootstrap sampling distribution (no hypothesize step)
4 boot_dist_ci <- los_tidy |>
5   specify(LOS ~ Group) |>
6   generate(reps = 5000, type = "bootstrap") |>
7   calculate(stat = "diff in means", order = c("Blunt", "Penetrating"))
8
9 # Percentile confidence interval
10 ci_infer <- get_confidence_interval(boot_dist_ci,
11                                     level = 0.95,
12                                     type = "percentile")
13 ci_infer
```

```
# A tibble: 1 × 2
  lower_ci upper_ci
  <dbl>    <dbl>
1  -0.142    4.50
```

```
1 # SE-based CI
2 ci_infer_se <- get_confidence_interval(boot_dist_ci,
3                                         level = 0.95,
4                                         type = "se",
5                                         point_estimate = obs_stat)
```

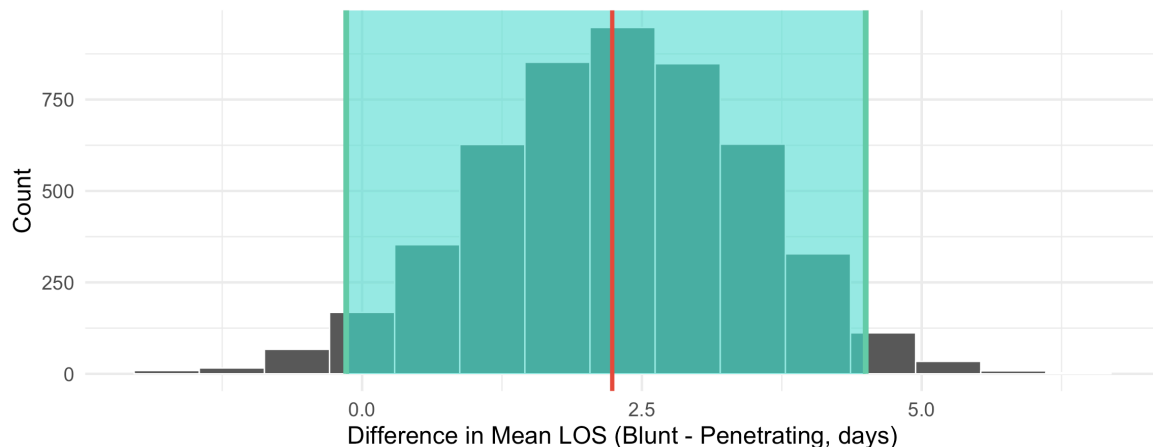


Visualizing the `infer` Bootstrap CI

```
1 visualize(boot_dist_ci) +  
2   shade_confidence_interval(ci_infer) +  
3   geom_vline(xintercept = obs_stat$stat,  
4             color = "#e74c3c", linewidth = 1.5) +  
5   labs(  
6     title      = "Bootstrap Sampling Distribution with 95% CI (via infer)",  
7     subtitle   = sprintf("Percentile CI: (%.3f, %.3f) | Observed = %.3f",  
8                          ci_infer$lower_ci, ci_infer$upper_ci, obs_stat$stat),  
9     x          = "Difference in Mean LOS (Blunt – Penetrating, days)",  
10    y          = "Count"  
11  )
```

Bootstrap Sampling Distribution with 95% CI (via infer)

Percentile CI: (-0.142, 4.500) | Observed = 2.235



Two-Sample Bootstrap Test with `infer`: A Limitation

⚠ `infer` does NOT implement the shift-bootstrap for two-sample tests

`generate(type = "bootstrap")` with `hypothesize(null = "independence")` resamples each group **separately with replacement** — exactly the same as the CI bootstrap. The null distribution is centered at the **observed difference**, not at 0. Calling `get_p_value()` on this distribution is essentially asking “what fraction of bootstrap samples are at least as extreme as the observed value?” — against a distribution centered *at* the observed value — which returns a meaningless p-value near 0.5 or higher.

The **shift-bootstrap** (C&H Ch. 8) must be implemented manually (as shown earlier in these slides). For two-sample hypothesis testing in `infer`, always use `type = "permute"`.

```
1 # Demonstrate the issue: bootstrap + hypothesize gives wrong null distribution
2 set.seed(551)
3 null_dist_boot_wrong <- los_tidy |>
4   specify(LOS ~ Group) |>
5   hypothesize(null = "independence") |>
6   generate(reps = 9999, type = "bootstrap") |>
7   calculate(stat = "diff in means", order = c("Blunt", "Penetrating"))
8
9 cat(sprintf("Center of bootstrap 'null' distribution: %.3f\n",
10            mean(null_dist_boot_wrong$stat)))
```

Center of bootstrap 'null' distribution: 2.243

```
1 cat(sprintf(" (should be ~0 under H0; instead equals observed diff)\n"))
  (should be ~0 under H0; instead equals observed diff)
```

```
1 cat(sprintf("Observed difference: %.3f\n", obs_stat$stat))
```

Paired Bootstrap with `infer`

For paired data, work directly with the **differences** as a one-sample problem:

```
1 set.seed(551)
2
3 # Bootstrap CI for mean difference (one-sample on differences)
4 boot_paired_infer <- bp_tidy |>
5   specify(response = diff) |>
6   generate(reps = 5000, type = "bootstrap") |>
7   calculate(stat = "mean")
8
9 ci_paired_infer <- get_confidence_interval(boot_paired_infer,
10                                           level = 0.95,
11                                           type = "percentile")
12
13 # Bootstrap test:  $H_0: \mu_D = 0$ 
14 obs_mean_D <- bp_tidy |>
15   specify(response = diff) |>
16   calculate(stat = "mean")
17
18 null_paired_infer <- bp_tidy |>
19   specify(response = diff) |>
20   hypothesize(null = "point", mu = 0) |>
21   generate(reps = 9999, type = "bootstrap") |>
```



infer Workflow Summary

► Code

Goal	specify()	hypothesize()	generate()	calculate()
Two-sample CI	Y ~ Group	(none)	bootstrap	diff in means/medians
Two-sample perm. test	Y ~ Group	independence	permute	diff in means/medians
Paired CI	response = diff	(none)	bootstrap	mean
Paired boot. test	response = diff	point, mu = 0	bootstrap	mean

infer Workflow Summary

► Code

Goal	specify()	hypothesize()	generate()	calculate()
Two-sample CI	Y ~ Group	(none)	bootstrap	diff in means/medians
Two-sample perm. test	Y ~ Group	independence	permute	diff in means/medians
Paired CI	response = diff	(none)	bootstrap	mean
Paired boot. test	response = diff	point, mu = 0	bootstrap	mean

When to Choose What

- **type = "permute"**: Two-sample hypothesis testing. Generates the null distribution by relabeling groups — the only correct **infer** approach for two-sample tests.
- **type = "bootstrap" without hypothesize()**: Constructing CIs (sampling distribution centered at observed statistic).
- **type = "bootstrap" with hypothesize(null = "point")**: One-sample bootstrap tests (shifts distribution to H_0 value). Works correctly for one-sample settings only.
- **Two-sample shift-bootstrap test**: Must be coded manually (see earlier section) — **infer** does not support it.

Comparing Methods: Simulation

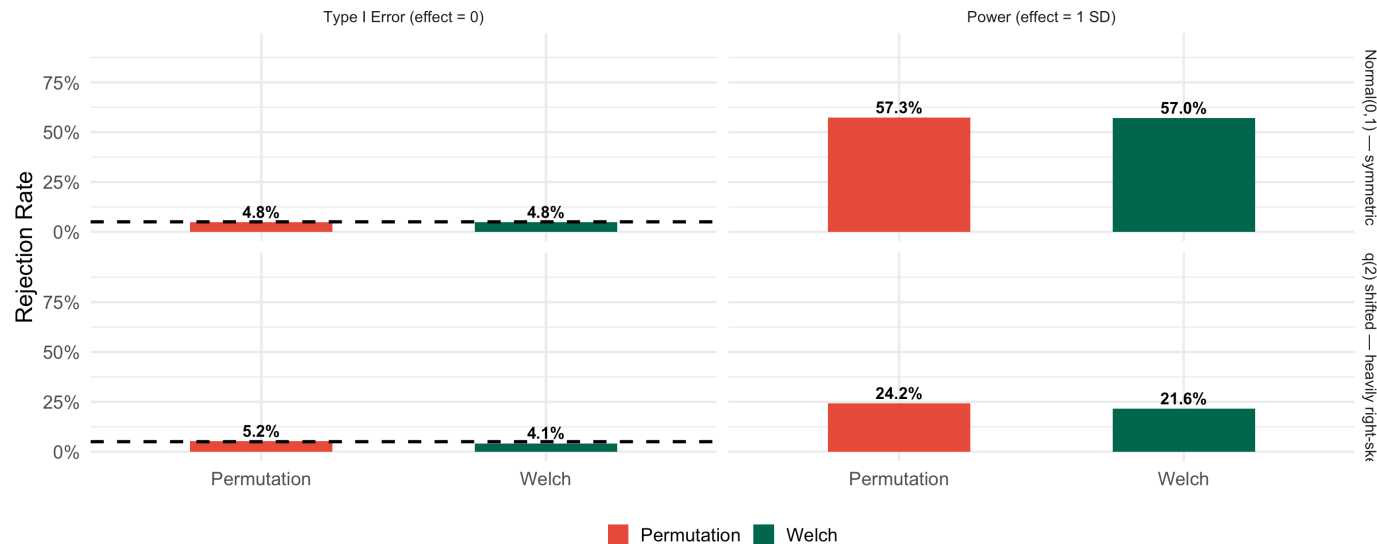
Type I Error and Power: Welch t-test vs. Permutation Test

The permutation test derives its null distribution empirically rather than from the t -distribution. For the **mean difference** statistic, how much does it matter?

► Code

Welch t-test vs. Permutation Test (n = 10 per group, 3000 simulations)

Both methods control Type I error near 5% and have similar power — permutation's advantage is flexibility.



i Note

The two-sided Welch t -test is **robust to skewness for Type I error** even at small n : skewness makes each tail individually wrong, but in a two-sided test the errors largely cancel. Permutation's advantage is **generality**: it works with *any* test statistic, not just means.

Key Takeaways

Key Takeaways (C&H Ch. 3, 5, 8; DBC 10.6)

! Bootstrap and Permutation for Two Samples

1. **Two-sample bootstrap CI:** Resample each group **independently** with replacement; construct percentile, basic, or SE CI. Works for any statistic. No distributional assumption.
2. **Two-sample bootstrap test:** Shift both samples to satisfy H_0 (equal grand mean), then resample to generate null distribution. Use when permutation assumptions are not met.
3. **Permutation test:** Pool all N observations, **relabel** groups at random; fraction of permuted statistics $\geq |T_{\text{obs}}|$ gives a valid p-value under $H_0 : F_1 = F_2$ (**exchangeability**). Most natural for randomized experiments.
4. **Permutation validity:** Requires **exchangeability** — groups can only differ in label, not in variance or shape, under H_0 . Fails for paired/clustered data and when groups differ on nuisance parameters.
5. **Paired bootstrap CI:** Resample **whole pairs/differences** to preserve pairing; CI for μ_D with no normality assumption.
6. **Paired bootstrap test:** Shift differences to mean 0 (imposing $H_0 : \mu_D = 0$), then resample; gives valid p-value without normality.
7. **infer package:** Unified specify $\rightarrow$ hypothesize $\rightarrow$ generate $\rightarrow$ calculate pipeline. Use `type = "permute"` for two-sample tests and `type = "bootstrap"` for CIs and one-sample tests. The shift-bootstrap for two-sample testing must be coded manually.

Method Decision Guide

Method Decision Guide for Two-Sample Inference

Setting	Two independent samples (any statistic)	Two independent samples (mean only)	Paired / matched data	Non-normal or small n
CI Method	Two-sample bootstrap CI	Welch t CI or bootstrap CI	Paired t CI or paired bootstrap CI	Bootstrap CI (percentile)
Test Method	Permutation test (if exchangeable) or bootstrap test	Welch t-test or permutation test	Paired t-test or paired bootstrap test	Bootstrap test

Looking Ahead

Next class (Lesson 21): Nonparametric Inference (DBC Ch. 14)

- Sign test: based on the binomial distribution, zero distributional assumptions
- Wilcoxon signed-rank test: uses ranks of $|D_i|$, requires symmetric differences
- Wilcoxon rank-sum test (Mann–Whitney): independent samples, ranks all N observations
- Asymptotic relative efficiency vs. parametric counterparts

References

- Chihara, L. M. and Hesterberg, T. C. *Mathematical Statistics with Resampling and R* (Wiley). Chapters 3, 5, and 8.
- Devore, Berk, and Carlton. *Modern Mathematical Statistics with Applications* (Springer). Section 10.6.
- Couch, S., Bray, A., et al. *infer: An R package for tidyverse-friendly statistical inference*. JOSS, 2021.
<https://infer.tidymodels.org>

Questions?

Thank you!